47 SLAJDÓW · GOTOWE WYPOWIEDZI · 47 PYTAŃ PROFESORA

Stanowisko
Co mówić podczas prezentacji

Gotowe wypowiedzi do 47 slajdów Adrian_Tur_1.pptx, krótkie warianty, przejścia i odpowiedzi na pytania profesora.

Wersja kodu: 1.0.9 (11) · gałąź android · analiza i weryfikacja: 6 listopada 2026

Jak korzystać ze scenariusza

Scenariusz odpowiada dokładnie **47 slajdom** pliku `/home/atur/Downloads/Adrian_Tur_1.pptx`, w kolejności zapisanej w prezentacji. Zawiera gotowe wypowiedzi w pierwszej osobie, krótkie warianty, przejścia i odpowiedzi na pytania profesora. Możesz uczyć się z całego modułu, a podczas wystąpienia korzystać tylko ze skrótu.

Wygodny widok: scenariusz HTML. **Do wydruku:** scenariusz PDF. **Na ostatnią powtórkę:** ściąga PDF.

Do prowadzenia pokazu: kopia PPTX z wypowiedziami w notatkach prelegenta. Slajdy mają zachowaną treść i kolejność; rozszerzone notatki są przeznaczone do widoku prowadzącego.

Jak korzystać ze scenariusza

Powiedz to wypowiedź do wygłoszenia, a nie opis tego, jak należałoby mówić. **Wersja krótka** zastępuje dłuższą wypowiedź, gdy masz mniej czasu. Nie czytaj obu wariantów jeden po drugim. **Przejście** łączy slajd z następnym. **Profesor pyta** i **Odpowiedź** służą przygotowaniu do dyskusji; nie są częścią podstawowego wystąpienia.

Przyjmij roboczo około **20–25 minut na wersję podstawową**, bez pełnej demonstracji na urządzeniu i bez pytań. To szacunek do sprawdzenia stoperem, nie zmierzony czas Twojej wypowiedzi. Przy krótszym limicie korzystaj z wariantów krótkich, a szczegóły techniczne rozwijaj po pytaniu. Demonstrację ze slajdu 39 zaplanuj osobno; restart tabletu ma zmienny czas.

Główna myśl całej prezentacji

Zapamiętaj ciąg: **problem urządzenia zadaniowego → konfiguracja administratora → polityki Androida → trwały stan → weryfikacja na urządzeniu**. Najczęściej wracające rozróżnienia to: instalacja i ochrona, zapis i zastosowanie, obecność kodu i wynik testu, lista nawigacji i filtr całej sieci.

Podstawa wypowiedzi

Tekst slajdów i notatki prelegenta odczytano z podanego PPTX. Twierdzenia o implementacji porównano z `/home/atur/Desktop/MYAPP`, commit `bb7dd3cf5de9124162a0544844600cced8a0a40`. Dane uczelni, tematu i osób oraz przywołania karty pracy pochodzą z prezentacji; sama karta nie jest załącznikiem do tego modułu. Oryginalnego PPTX nie zmieniano.

Prezentacja opisuje stan z 5 listopada 2026 r. Wcześniejsza dokumentacja zawiera dodatkowe sprawdzenia lokalne z 6 listopada: udane budowanie, lint i 62 sprawdzenia wybranej logiki. Możesz o nich powiedzieć jako o **dodatkowym zakresie weryfikacji**, opisanym w rozdziale 10. Nie zastępują prób ochrony na fizycznym tablecie.

Gdzie wrócić po wyjaśnienia

Slajdy	Temat do powtórzenia
1–18	Cel projektu, decyzje i ograniczenia
19–24, 29–31, 40	Architektura, dane, budowanie
25–27, 32–33, 37	Zapis, zastosowanie, sesje i odtwarzanie
34–36	PIN i uprawnienia, WebView
38–47	Procedury i próby, wyniki weryfikacji

Slajd 01 — STANOWISKO

Cel: Przedstaw temat i od razu powiedz, jaki problem rozwiązuje aplikacja.

Powiedz:

> Dzień dobry, nazywam się Adrian Tur. Przedstawię projekt aplikacji mobilnej Stanowisko, która tworzy kontrolowane środowisko pracy na Androidzie. Administrator określa dostępne aplikacje i stronę internetową, a systemowe polityki ograniczają korzystanie z urządzenia do przygotowanego zakresu. W prezentacji pokażę założenia, najważniejsze mechanizmy implementacji oraz sposób weryfikowania ich działania.

Wersja krótka:

> Przedstawiam Stanowisko — aplikację Android do udostępniania wybranych programów i strony WWW pod kontrolą administratora.

Przejście: Najpierw krótko pokażę układ prezentacji.

Profesor pyta: Co oznacza „zdefiniowany komponent” w tym projekcie?

Odpowiedź: W implementacji chodzi przede wszystkim o wybraną aplikację identyfikowaną pakietem oraz wbudowaną stronę WWW. Nie deklaruję filtrowania każdej funkcji i każdego ekranu wewnątrz aplikacji zewnętrznej.

Slajd 02 — Trzy etapy projektu

Cel: Uporządkuj oczekiwania słuchacza.

Powiedz:

> Prezentację podzieliłem na trzy części. Najpierw wyjaśnię problem, zakres i wymagania. Następnie przejdę do implementacji: pulpitu, polityk Androida, danych oraz PIN-u. Na końcu pokażę scenariusz demonstracji, stan weryfikacji i dalsze kroki potrzebne przed wdrożeniem.

Wersja krótka:

> Omówię założenia, implementację i weryfikację, a na końcu kierunki rozwoju.

Przejście: Zacznę od tego, dlaczego zwykły dostęp do Androida bywa niewłaściwy dla stanowiska zadaniowego.

Profesor pyta: Która część jest głównym rezultatem pracy?

Odpowiedź: Aplikacja mobilna z lokalną konfiguracją i implementacją polityk urządzenia. Plan badań i procedura wdrożenia pozwalają następnie ocenić jej działanie w konkretnym środowisku.

Slajd 03 — Urządzenie do konkretnego zadania

Cel: Wyjaśnij problem, zanim wymienisz technologie.

Powiedz:

> Tablet przeznaczony do jednego zadania nie zawsze powinien udostępniać użytkownikowi cały system. Możliwość uruchamiania dowolnych aplikacji albo zmiany ustawień może zakłócać pracę stanowiska. Przykładem jest ekran informacyjny, który powinien stale prezentować wskazaną treść. Problemem projektowym jest więc takie ograniczenie środowiska, aby użytkownik miał potrzebne funkcje, a administrator zachował możliwość obsługi urządzenia.

Wersja krótka:

> Urządzenie zadaniowe powinno udostępniać potrzebne funkcje i utrzymywać konfigurację ustaloną przez administratora.

Przejście: Odpowiedzią jest połączenie listy dozwolonych komponentów z mechanizmami systemowymi.

Profesor pyta: Czy ukrycie ikon rozwiązałoby ten problem?

Odpowiedź: Sam wygląd pulpitu nie egzekwuje ograniczeń systemowych. Potrzebny jest mechanizm kontrolujący zadania i dostępne pakiety, dlatego aplikacja korzysta z polityk Androida.

Slajd 04 — Administrator ustala granice

Cel: Powiąż trzy elementy rozwiązania z problemem.

Powiedz:

> Koncepcja składa się z trzech części. Administrator wybiera dostępne aplikacje i ewentualną stronę. Device Owner i Lock Task pozwalają zastosować ograniczenia w Androidzie. PIN chroni zmianę konfiguracji i autoryzowany powrót z sesji pojedynczej. Istotna jest kolejność: samo zainstalowanie APK i zapisanie listy nie uruchamia jeszcze pełnej ochrony. Politykę trzeba osobno zastosować.

Wersja krótka:

> Administrator wybiera zakres, Android egzekwuje politykę, a PIN chroni administrację. Zapis konfiguracji i jej zastosowanie są osobnymi krokami.

Przejście: Na tej koncepcji opierają się cel pracy i pięć tez.

Profesor pyta: Czy Device Owner i PIN to ten sam rodzaj uprawnienia?

Odpowiedź: Nie. Device Owner jest rolą nadawaną przez Androida. PIN jest mechanizmem autoryzacji wewnątrz Stanowiska, który pozwala korzystać z funkcji administratora aplikacji.

Slajd 05 — Cel pracy i pięć tez

Cel: Przedstaw tezy jako przedmiot sprawdzenia.

Powiedz:

> Celem jest przygotowanie środowiska z dostępem do zdefiniowanych komponentów. Pięć tez dotyczy skuteczności ograniczeń, bezpieczeństwa i przewidywalności, elastyczności listy, użycia oficjalnych API oraz nakładu pracy administratora. Implementacja pokazuje, jak te założenia realizować. Żeby uznać tezy za potwierdzone, potrzebne są jeszcze odpowiednie próby i pomiary, do których wrócę w końcowej części.

Wersja krótka:

> Tezy obejmują kontrolę środowiska, bezpieczeństwo, elastyczność, oficjalne API i wygodę administracji. Ich skuteczność wymaga weryfikacji.

Przejście: Najpierw pokażę architekturę, która ma umożliwiać realizację tych założeń.

Profesor pyta: Czy samo istnienie kodu potwierdza tezę o bezpieczeństwie?

Odpowiedź: Potwierdza obecność zaprojektowanych mechanizmów. Ich skuteczność wymaga prób określonych zagrożeń, np. nieautoryzowanego opuszczenia stanowiska, na przygotowanym urządzeniu.

Slajd 06 — Lokalny obieg konfiguracji

Cel: Przeprowadź słuchacza po schemacie w kolejności przepływu.

Powiedz:

> Zmiana zaczyna się w panelu administratora. ViewModel udostępnia stan i obsługuje akcje interfejsu, a PolicyEngine wykonuje operacje dotyczące polityki. Silnik współpracuje z Androidem oraz lokalnym magazynem danych. Room przechowuje politykę i audyt, a pozostałe ustawienia trafiają do preferences. Ten obieg jest lokalny. Osobną zależność od sieci ma treść wyświetlana przez WebView.

Wersja krótka:

> Interfejs przekazuje akcje do logiki polityk, która współpracuje z Androidem i lokalnymi danymi. Sieci potrzebuje wybrana strona WWW.

Przejście: Zanim przejdę do zastosowań, wskażę źródła wiedzy wykorzystane w prezentacji.

Profesor pyta: Czy jest to architektura klient–serwer?

Odpowiedź: Lokalna konfiguracja i polityki nie wymagają serwera aplikacji. WebView jest klientem skonfigurowanej strony, ale nie oznacza to istnienia centralnego backendu zarządzającego Stanowiskiem.

Slajd 07 — Źródła i bibliografia

Cel: Wyjaśnij rolę źródeł, bez czytania wszystkich pozycji.

Powiedz:

> Bibliografia obejmuje publikacje dotyczące Androida, bezpieczeństwa i Kotlina, a także wskazane w prezentacji prace inżynierskie. Osobną grupą są materiały Android Developers dotyczące kontrolera polityk i urządzeń dedykowanych. Książki służą uporządkowaniu podstaw, natomiast szczegóły działania konkretnych API trzeba odnosić do dokumentacji platformy i wersji użytej w projekcie.

Wersja krótka:

> Podstawą są publikacje o Androidzie i Kotlinie oraz dokumentacja Android Developers dotycząca polityk urządzenia.

Przejście: Przejdę teraz od podstaw do przykładowych zastosowań aplikacji.

Profesor pyta: Czy pozycja o bezpieczeństwie z 2014 roku wystarcza do opisu współczesnego Androida?

Odpowiedź: Może stanowić źródło podstaw architektury. Aktualne kontrakty DevicePolicyManager, Lock Task czy WebView trzeba sprawdzać w dokumentacji odpowiednich API. Nie należy przypisywać starszej książce opisu nowych funkcji.

Slajd 08 — Jedna aplikacja, różne stanowiska

Cel: Pokaż trzy zastosowania tego samego mechanizmu.

Powiedz:

> Ta sama konstrukcja może obsłużyć różne zadania. W kiosku informacyjnym udostępniam stronę. Na tablecie służbowym przygotowuję zestaw aplikacji. Na stanowisku demonstracyjnym uruchamiam jedną aplikację, a powrót chronię PIN-em. Są to scenariusze użycia wynikające z konfiguracji, a nie trzy osobno napisane produkty ani lista zakończonych wdrożeń.

Wersja krótka:

> Zmieniając konfigurację, mogę przygotować kiosk WWW, zestaw aplikacji służbowych albo demonstrację jednej aplikacji.

Przejście: Takie scenariusze trzeba następnie sprawdzić w konkretnym środowisku pilotażowym.

Profesor pyta: Co decyduje o scenariuszu: nowy APK czy konfiguracja?

Odpowiedź: W tych przykładach przede wszystkim lista pakietów, konfiguracja strony i tryb uruchamiania. Nowa funkcja wykraczająca poza te możliwości wymagałaby zmiany kodu.

Slajd 09 — Pilotaż w środowisku testowym

Cel: Określ realny status miejsca wdrożenia.

Powiedz:

> Jako możliwe środowisko pilotażu prezentacja wskazuje uczelnię lub lokalne laboratorium. Pilotaż powinien ustalić docelowy model urządzenia, wersję Androida i zasady organizacji. Wtedy można ocenić zachowanie ochrony oraz wygodę obsługi. Ten slajd opisuje proponowane miejsce prób; nie jest informacją o zakończonym wdrożeniu produkcyjnym w uczelni.

Wersja krótka:

> Pilotaż planuję w środowisku testowym, po określeniu modelu urządzenia i zasad jego użycia.

Przejście: Żeby taki pilotaż był wykonalny, trzeba jasno określić zakres informatyczny pracy.

Profesor pyta: Jaki tablet został już zatwierdzony do wdrożenia?

Odpowiedź: Prezentacja nie potwierdza zatwierdzenia konkretnego modelu. Dobór sprzętu i sprawdzenie jego zachowania są elementami pilotażu.

Slajd 10 — Zakres informatyczny

Cel: Oddziel zadania projektu od przyszłych rozszerzeń.

Powiedz:

> Zakres obejmuje natywne programowanie Androida, interfejs Compose, lokalne dane, polityki urządzenia i kontrolę dostępu. Ważna jest również czytelność konfiguracji oraz przygotowanie scenariuszy prób. Centralne zarządzanie flotą pozostaje osobnym etapem. Dzięki temu projekt skupia się na działaniu pojedynczego stanowiska i jego integracji z systemem.

Wersja krótka:

> Zakres to aplikacja Android, lokalna konfiguracja i polityki urządzenia. Centralne zarządzanie flotą pozostaje poza bieżącą implementacją.

Przejście: Konkretnym rezultatem tego zakresu jest aplikacja mobilna.

Profesor pyta: Dlaczego nie dołączono od razu panelu centralnego?

Odpowiedź: Wymagałby dodatkowo serwera, komunikacji, uwierzytelnienia urządzeń i obsługi synchronizacji. To odrębny zakres, który nie jest potrzebny do przedstawienia lokalnego mechanizmu stanowiska.

Slajd 11 — Dziełem jest aplikacja mobilna

Cel: Nazwij rezultat i jego części.

Powiedz:

> Głównym rezultatem jest lokalnie konfigurowana aplikacja Android. Łączy pulpit użytkownika, panel administratora, dwa sposoby pracy i trwały zapis konfiguracji. Dokumentacja oraz instalatory wspierają przygotowanie urządzenia. Ewentualny panel webowy stanowi osobny komponent; omawiana tutaj aplikacja mobilna nie zawiera centralnego API zarządzania.

Wersja krótka:

> Rezultatem jest aplikacja mobilna z pulpitem, politykami, PIN-em, danymi lokalnymi i instrukcją przygotowania urządzenia.

Przejście: Z tych funkcji korzystają osoby o różnych odpowiedzialnościach.

Profesor pyta: Czy strona otwierana w WebView jest panelem administratora?

Odpowiedź: Nie musi nim być. Jest treścią wskazaną w konfiguracji. Administrację Stanowiska realizuje lokalny interfejs chroniony PIN-em.

Slajd 12 — Użytkownicy i odpowiedzialność

Cel: Wyjaśnij role organizacyjne i ich odpowiedniki w aplikacji.

Powiedz:

> Administrator przygotowuje listę komponentów, PIN i politykę. Operator korzysta z udostępnionych funkcji. Organizacja ustala przeznaczenie urządzenia, a dział IT odpowiada za sprzęt i aktualizacje. W samej aplikacji nie są to cztery rodzaje kont w bazie. Praktyczny podział dostępu polega na zwykłej obsłudze pulpitu i czasowej autoryzacji administratora przez PIN.

Wersja krótka:

> Operator pracuje na pulpicie, administrator ustala konfigurację, a organizacja i IT odpowiadają za zasady oraz przygotowanie sprzętu.

Przejście: Następnie ocenię, czy projekt jest wykonalny przy przyjętych narzędziach i warunkach.

Profesor pyta: Gdzie przechowywane są konta administratorów?

Odpowiedź: Ten projekt nie ma bazy kont i przypisanych ról użytkowników. Weryfikuje jedno lokalne poświadczenie PIN, a autoryzacja administratora jest krótkotrwałym stanem procesu.

Slajd 13 — Wykonalność projektu

Cel: Uzasadnij możliwość realizacji i właściwie odczytaj wartość „0 zł”.

Powiedz:

> Wykonalność techniczna opiera się na Kotlinie, Compose, Room i oficjalnych API Androida. Minimalna obsługiwana wersja to Android 14. Emulator pomaga w rozwoju, natomiast ochronę trzeba sprawdzić na sprzęcie. Wartość zero złotych odnosi się na slajdzie do opłat za wybrany stos narzędzi. Nie jest budżetem całego przedsięwzięcia, bo pozostają koszty urządzenia, przygotowania, testów i pracy.

Wersja krótka:

> Stos pozwala zrealizować projekt na Androidzie 14+. „0 zł” nie obejmuje sprzętu ani nakładu pracy i prób.

Przejście: Realizację podzielono na sześć następujących po sobie etapów.

Profesor pyta: Czy aplikacja nie generuje żadnych kosztów wdrożenia?

Odpowiedź: Ten slajd tego nie wykazuje. Wdrożenie wymaga wyceny sprzętu, pracy, utrzymania i prób. Kwota na slajdzie nie jest analizą całkowitego kosztu.

Slajd 14 — Sześć etapów realizacji

Cel: Pokaż zależność prac zamiast odczytywać listę.

Powiedz:

> Kolejność prac wynika z zależności między częściami aplikacji. Najpierw określam wymagania i dane, następnie buduję pulpit i integruję polityki Androida. Dopiero wtedy można oceniać zachowanie ograniczeń i restartu. Ostatni etap obejmuje pakiet instalacyjny oraz instrukcję. Jest to plan etapów, bez przypisywania im nieudokumentowanych dat czy pracochłonności.

Wersja krótka:

> Najpierw analiza i dane, potem pulpit oraz polityki, a następnie próby i przygotowanie dystrybucji.

Przejście: Przy takiej realizacji trzeba przewidzieć najważniejsze ryzyka.

Profesor pyta: Czy wszystkie etapy są już zakończone?

Odpowiedź: Kod i materiały instalacyjne są dostępne. Pełne próby ochrony, pomiary i ocena wdrożeniowa na docelowym sprzęcie pozostają odrębnym zadaniem.

Slajd 15 — Ryzyka wymagają prób na sprzęcie

Cel: Wybierz najważniejsze ryzyka i wskaż sposób ich ograniczania.

Powiedz:

> Największe ryzyka dotyczą przygotowania Device Owner, różnic producentów i możliwości nieautoryzowanego opuszczenia środowiska. Odpowiedzią są kontrole warunków wejścia oraz próby na docelowym urządzeniu. Pozostałe ryzyka obejmują konfigurację i aktualizację danych. Oceny „średnie” czy „wysokie” są tutaj jakościową oceną projektową; nie pochodzą z policzonej częstości awarii.

Wersja krótka:

> Główne ryzyka to przygotowanie urządzenia, różnice OEM i skuteczność ograniczeń. Ich ocenę trzeba uzupełnić próbami sprzętowymi.

Przejście: Z ryzyk wynikają granice obecnego rozwiązania.

Profesor pyta: Na jakiej próbie oszacowano prawdopodobieństwo jako średnie?

Odpowiedź: Nie jest to estymacja statystyczna. Tabela porządkuje ryzyka jakościowo, żeby wskazać priorytety sprawdzenia. Dane ilościowe wymagałyby określenia urządzeń, scenariuszy i liczby powtórzeń.

Slajd 16 — Granice rozwiązania

Cel: Opisz granice jako warunki użytkowania.

Powiedz:

> Aplikacja zarządza lokalnym urządzeniem, więc nie zastępuje centralnego systemu obsługi floty. Pełna ochrona wymaga przygotowania roli Device Owner i zastosowania polityki. Zachowanie zależy także od wersji Androida i modyfikacji producenta. Pulpit oraz dane działają lokalnie, natomiast strona WWW potrzebuje dostępności swojego serwera i odpowiedniej łączności.

Wersja krótka:

> Rozwiązanie jest lokalne, wymaga przygotowanego Androida, a działanie WebView zależy od dostępności treści.

Przejście: W tych granicach określono dziesięć wymagań funkcjonalnych.

Profesor pyta: Czy wszystko będzie działało bez internetu?

Odpowiedź: Lokalny pulpit, konfiguracja i mechanizm polityki nie wymagają centralnego serwera. Nie gwarantuję pracy strony WWW bez dostępu do jej treści; projekt nie ma własnego systemu synchronizacji offline tej strony.

Slajd 17 — Dziesięć wymagań funkcjonalnych

Cel: Zgrupuj wymagania, zamiast czytać dziesięć wierszy.

Powiedz:

> Wymagania można podzielić na trzy grupy. Pierwsza obejmuje pulpit, aplikacje i stronę. Druga dotyczy administracji: PIN-u, polityk i uprawnień. Trzecia to utrzymanie stanu, ustawienia ekranu, audyt oraz kontrolowane wyjście. Później pokażę, które elementy kodu odpowiadają każdemu wymaganiu.

Wersja krótka:

> Wymagania obejmują funkcje użytkownika, kontrolę administratora i utrzymanie środowiska po zmianach oraz restartach.

Przejście: Oprócz funkcji potrzebne są kryteria jakości działania.

Profesor pyta: Jak stwierdzić, że wymaganie zostało spełnione?

Odpowiedź: Najpierw wskazuję implementację, a następnie definiuję scenariusz akceptacyjny i oczekiwany wynik. Dla ochrony systemowej konieczny jest odczyt rzeczywistego stanu urządzenia.

Slajd 18 — Jakość środowiska pracy

Cel: Rozróżnij wymagania jakościowe i udowodnione wyniki.

Powiedz:

> Wymagania niefunkcjonalne dotyczą zgodności platformy, lokalności, odtwarzania konfiguracji, kontroli dostępu i utrzymywalności. Szczególnie ważna jest spójność: administrator ma rozróżniać konfigurację zapisaną i zastosowaną. Część tych właściwości widać w strukturze kodu. Ich zachowanie w warunkach awarii i na różnych urządzeniach wymaga osobnych prób.

Wersja krótka:

> Istotne są zgodność, lokalna praca, odtwarzanie i czytelny stan konfiguracji. Jakość działania trzeba ocenić testami.

Przejście: Do realizacji tych wymagań dobrano następujący stos technologiczny.

Profesor pyta: Czy podano limit czasu uruchomienia aplikacji?

Odpowiedź: Prezentacja nie zawiera potwierdzonego wyniku ani konkretnego progu akceptacji czasu startu. Trzeba zdefiniować sposób pomiaru i oczekiwaną wartość przed pilotażem.

Slajd 19 — Stos technologiczny

Cel: Powiąż narzędzie z zadaniem, które wykonuje.

Powiedz:

> Kotlin służy do implementacji logiki. Compose i Material 3 tworzą interfejs, a Room zapewnia warstwę nad lokalnym SQLite. Korutyny i Flow obsługują zadania oraz obserwowanie zmian. Najważniejszą integrację systemową zapewnia DevicePolicyManager, a Keystore chroni klucz poświadczenia PIN-u. Gradle buduje APK, natomiast ADB i instalatory pomagają przygotować urządzenie. SDK 36 to poziom kompilacji; minimalny poziom instalacji wynosi 34.

Wersja krótka:

> Kotlin i Compose tworzą aplikację, Room przechowuje stan, a DevicePolicyManager realizuje kontrolę urządzenia. Gradle buduje, ADB instaluje.

Przejście: Zobaczmy teraz, jak te technologie łączą się w architekturze aplikacji.

Profesor pyta: Dlaczego compileSdk 36 nie oznacza wymogu Androida 16?

Odpowiedź: O minimalnym systemie decyduje minSdk, tutaj 34. compileSdk określa API dostępne kompilatorowi, a targetSdk docelowy poziom zachowań platformy. To trzy odrębne ustawienia.

Slajd 20 — PolicyEngine łączy interfejs z systemem

Cel: Omów centralną odpowiedzialność silnika.

Powiedz:

> Centralnym elementem operacji jest PolicyEngine. Otrzymuje żądania dotyczące zapisu, zastosowania i uruchamiania sesji. Z jednej strony czyta lub aktualizuje lokalny stan, z drugiej wywołuje usługi Androida. Dzięki temu sama kontrolka w interfejsie nie jest mechanizmem blokady. Ograniczenia wynikają z polityki systemowej, a UI prezentuje ich stan i udostępnia dozwolone działania.

Wersja krótka:

> PolicyEngine przekłada działania administratora na lokalny zapis i operacje systemowe Androida.

Przejście: Pozostałe części mają własne, bardziej szczegółowe odpowiedzialności.

Profesor pyta: Czy to ścisła Clean Architecture?

Odpowiedź: To praktyczny podział odpowiedzialności z elementami MVVM. Ekran sięga także do usług aplikacji, więc nie przedstawiam go jako pełnej izolacji warstw według formalnego wzorca Clean Architecture.

Slajd 21 — Podział odpowiedzialności

Cel: Wyjaśnij, co oznaczają warstwy na slajdzie.

Powiedz:

> Interfejs odpowiada za wyświetlanie i wprowadzanie danych. ViewModel udostępnia obserwowany stan oraz obsługuje operacje ekranu. Silnik i walidatory realizują reguły działania. Magazyny utrwalają politykę, PIN oraz ustawienia strony. Ten podział ułatwia znalezienie miejsca zmiany: innego przy poprawce wyglądu, innego przy zmianie polityki urządzenia.

Wersja krótka:

> Oddzielono prezentację, stan, logikę i dane, dzięki czemu każda część ma określone zadanie.

Przejście: W silniku można wskazać cztery podstawowe operacje.

Profesor pyta: Ile modułów Gradle ma projekt?

Odpowiedź: Jeden moduł aplikacji, `:app`. Nazwy „moduł” i „warstwa” na tych slajdach opisują podział logiczny, a nie osobne moduły budowania dla każdego pliku.

Slajd 22 — Cztery operacje polityki

Cel: Odróżnij zapis, zastosowanie, uruchomienie i serwis.

Powiedz:

> `save` utrwala konfigurację administratora. `apply` próbuje zastosować ją w Androidzie i potwierdzić stan ochrony. `launch` uruchamia wskazany dozwolony program, a `serviceExit` zdejmuje ograniczenia w trybie serwisowym. Są to lokalne metody Kotlinu. Warunki w tabeli są skrótem: zastosowanie wymaga także między innymi braku aktywnej sesji i dostępnej nakładki.

Wersja krótka:

> Zapis, zastosowanie, start programu i serwis to osobne operacje, z osobnymi warunkami wejścia.

Przejście: Żeby rozdzielić ich skutki, potrzebny jest odpowiedni model danych.

Profesor pyta: Czy serviceExit odbiera aplikacji rolę Device Owner?

Odpowiedź: Nie. Wyłącza określone ograniczenia i zamiar odtwarzania ochrony. Rola właściciela, PIN i zapisane ustawienia pozostają; nie jest to pełne wycofanie urządzenia z zarządzania.

Slajd 23 — Dwie encje i ustawienia lokalne

Cel: Rozróżnij bazę polityki, dziennik i magazyny ustawień.

Powiedz:

> Room ma dwie encje. PolicyRecord przechowuje konfigurację zapisaną i ostatnio zastosowaną, a AuditEvent czas, typ i szczegóły zdarzenia. Lista reguł jest zapisana jako JSON. Drobniejsze ustawienia strony i odtwarzania trafiają do SharedPreferences. PinStore przechowuje zaszyfrowane poświadczenie w preferences i korzysta z klucza Android Keystore. Na slajdzie nie ma relacji klucza obcego między tymi encjami.

Wersja krótka:

> PolicyRecord przechowuje politykę, AuditEvent dziennik. Pozostałe ustawienia są w preferences, a klucz poświadczenia w Keystore.

Przejście: Przyjrzyjmy się teraz formatowi pojedynczej listy reguł.

Profesor pyta: Czy PIN jest trzecią tabelą Room?

Odpowiedź: Nie. PinStore ma osobny magazyn preferences i klucz Keystore. Dwie encje Room dotyczą polityki i audytu.

Slajd 24 — Konfiguracja jest danymi

Cel: Wytłumacz JSON na prostym przykładzie.

Powiedz:

> W przykładzie każdy wpis zawiera nazwę pakietu i pole `single`. Wartość true oznacza sesję jednej aplikacji, false uruchomienie w trybie wielu aplikacji. Nazwy pakietów na slajdzie są fikcyjne. Przed przyjęciem konfiguracji aplikacja sprawdza dostępność pakietów i duplikaty. Niezależnie waliduje adresy strony. Sam poprawny JSON nie oznacza jeszcze poprawnej ani zastosowanej polityki.

Wersja krótka:

> Reguła wskazuje pakiet i tryb sesji. Konfiguracja jest walidowana, zapisywana i dopiero później stosowana.

Przejście: Taki podział widać w całej ścieżce pracy administratora.

Profesor pyta: Dlaczego w JSON-ie nie ma ikony i nazwy aplikacji?

Odpowiedź: Pakiet jest identyfikatorem reguły. Etykietę i ikonę aplikacja odczytuje z PackageManager na urządzeniu, więc nie musi dublować ich w dokumencie konfiguracji.

Slajd 25 — Zmiana polityki krok po kroku

Cel: Pokaż świadome rozdzielenie decyzji administratora i skutku systemowego.

Powiedz:

> Administrator najpierw potwierdza tożsamość PIN-em, wybiera komponenty i zapisuje nową rewizję. Następnie osobno stosuje konfigurację. Dzięki temu można przygotować zmianę bez natychmiastowej zmiany aktywnego środowiska. Błędna konfiguracja jest odrzucana. Dla PIN-u wprowadzono ograniczanie kolejnych prób: narastające opóźnienie zaczyna się od piątej nieudanej weryfikacji.

Wersja krótka:

> Ścieżka administratora to PIN, wybór, zapis i zastosowanie. Zapisana zmiana może jeszcze czekać na aktywację.

Przejście: Najważniejszy z tych kroków, zastosowanie, ma własną sekwencję kontroli.

Profesor pyta: Czy pierwszy błędny PIN blokuje dostęp na dłuższy czas?

Odpowiedź: Nie. Pierwsze cztery niepowodzenia nie dodają opóźnienia. Po piątym jest ono równe 30 sekund, potem rośnie i osiąga limit 15 minut.

Slajd 26 — Zapis nie uruchamia ochrony

Cel: Objaśnij kolejność od kontroli do potwierdzenia sukcesu.

Powiedz:

> `apply` sprawdza autoryzację i przygotowanie urządzenia, następnie ustawia listę pakietów oraz funkcje Lock Task. Aplikacja czeka na systemowy stan LOCKED i potwierdzenie gotowości nakładki. Dopiero po tych kontrolach zapisuje zastosowaną rewizję oraz aktywną politykę. Jeśli zmiana się nie powiedzie, kod podejmuje próbę przywrócenia poprzedniego stanu. Nie jest to jedna atomowa transakcja obejmująca Androida i bazę.

Wersja krótka:

> Najpierw zmiana i kontrola stanu Androida, potem zapis sukcesu. Samo utrwalenie listy nie aktywuje ochrony.

Przejście: Po zastosowaniu polityki użytkownik może uruchamiać przygotowane komponenty.

Profesor pyta: Czy błąd zawsze całkowicie cofa wszystkie zmiany?

Odpowiedź: Kod ma ścieżkę kompensacji w apply, ale także ona może napotkać błąd. Dlatego trzeba sprawdzić zapisany komunikat, audyt i rzeczywisty stan DPM, zamiast zakładać gwarantowane wycofanie.

Slajd 27 — Uruchomienie dozwolonego komponentu

Cel: Opisz przebieg w chronionym trybie i jego granice.

Powiedz:

> Ten schemat pokazuje uruchomienie w chronionym środowisku. Silnik znajduje regułę dla pakietu i sprawdza przygotowanie uprawnień. Dla pojedynczej sesji dodatkowo zawęża listę oraz zapisuje znacznik sesji. Następnie otwiera program i rejestruje zdarzenie. Zwykły powrót z sesji pojedynczej wymaga PIN-u. W kodzie istnieje też uruchamianie zwykłego programu bez aktywnej ochrony, ale nie daje ono pełnej kontroli stanowiska.

Wersja krótka:

> Chronione uruchomienie wymaga dozwolonej reguły i przygotowanych uprawnień. Pojedyncza sesja zawęża środowisko i chroni powrót PIN-em.

Przejście: Po tym przebiegu przejdę do obecnego stanu implementacji.

Profesor pyta: Czy każda aplikacja zawsze wymaga aktywnego Lock Task do startu?

Odpowiedź: Nie. Dla reguły wielu aplikacji silnik dopuszcza zwykłe uruchomienie przy wyłączonej ochronie. Pojedyncza sesja wymaga pełnej ochrony, a kafelek strony również wymaga chronionego stanu.

Slajd 28 — Stanowisko 1.0.9

Cel: Oddziel numer wydania od dowodu działania na sprzęcie.

Powiedz:

> Obecny kod opisuje wersję 1.0.9 dla Androida 14 i nowszego. Są dwa tryby pracy oraz dwie encje Room w schemacie wersji trzeciej. To wydanie rozwojowe. Sam numer wersji i dostępność kodu nie oznaczają zakończenia badań ochrony na docelowym sprzęcie.

Wersja krótka:

> Dostępna jest implementacja 1.0.9. Status kodu i status prób sprzętowych pozostają odrębne.

Przejście: Pokażę teraz, gdzie te mechanizmy znajdują się w projekcie.

Profesor pyta: Co oznacza 1.0.9 w porównaniu ze schematem v3?

Odpowiedź: 1.0.9 to wersja produktu. Schemat 3 opisuje strukturę bazy Room. Dodatkowo versionCode wynosi 11, a rewizja polityki zmienia się przy zapisie konfiguracji. Te numery mają różne znaczenie.

Slajd 29 — Struktura projektu

Cel: Wskaż najważniejsze pliki i nie czytaj całego drzewa.

Powiedz:

> Manifest deklaruje komponenty aplikacji. MainActivity otwiera główny ekran, WorkspaceScreen tworzy pulpit i panel, a ViewModel udostępnia ich stan. PolicyEngine zawiera operacje polityk. Modele danych, PIN i WebView mają własne pliki. Taki podział pozwala śledzić operację od przycisku przez logikę do zapisu lub wywołania Androida.

Wersja krótka:

> Najważniejsze obszary kodu to ekran, ViewModel, silnik polityk oraz osobne części danych, PIN-u i WWW.

Przejście: Kod trzeba następnie zbudować do postaci pakietu Androida.

Profesor pyta: Gdzie zaczyna się wykonywanie programu, skoro nie ma main?

Odpowiedź: Komponenty uruchamia Android zgodnie z manifestem. Tworzy Application, a potem odpowiednią Activity, usługę lub receiver. Własny kod reaguje metodami cyklu życia.

Slajd 30 — Budowanie i instalacja

Cel: Wyjaśnij polecenie i wynik budowania.

Powiedz:

> Polecenie assembleDebug uruchamia budowanie wariantu rozwojowego przez wrapper Gradle. Wynikiem jest APK pod wskazaną ścieżką. Konfiguracja rozróżnia minimalne API 34 oraz kompilację i target 36. Następnie ADB pozwala zainstalować pakiet na urządzeniu. Do uruchomienia Gradle potrzebne są poprawnie wskazane JDK i SDK. Taki build nie jest jeszcze gotowym, produkcyjnie podpisanym wydaniem.

Wersja krótka:

> Gradle tworzy debug APK, a ADB instaluje go na urządzeniu. Budowanie i przygotowanie produkcyjnej dystrybucji to osobne zadania.

Przejście: Pierwszą częścią implementacji, którą omówię dokładniej, jest trwały zapis danych.

Profesor pyta: Czy ten kod udało się zbudować?

Odpowiedź: W dodatkowej weryfikacji z 6 listopada wykonano assembleDebug i lintDebug z wynikiem BUILD SUCCESSFUL. Lint zgłosił 28 ostrzeżeń, bez błędów. To wynik budowania i analizy statycznej, nie test działania kiosku na tablecie.

Slajd 31 — Room przechowuje politykę i audyt

Cel: Wskaż pola, które wyrażają rozdzielenie zapisu i zastosowania.

Powiedz:

> W PolicyRecord warto zauważyć pary revision i appliedRevision oraz rules i appliedRules. Pierwsze pola opisują zapis administratora, drugie ostatnie udane zastosowanie. AuditEvent utrwala zdarzenia. Migracja 1 do 2 dodaje ustawienia niewygaszania, a 2 do 3 ustawienia pasków. Dzięki jawnej ścieżce migracji aktualizacja nie musi kasować wcześniejszej bazy, ale zachowanie migracji nadal wymaga sprawdzenia testem.

Wersja krótka:

> Room rozdziela zapisane i zastosowane reguły, przechowuje audyt oraz ma migracje ustawień ekranu i pasków.

Przejście: Dane opisują zamiar, natomiast egzekwowanie wykonują polityki Androida.

Profesor pyta: Czy appliedRevision zawsze dowodzi, że ochrona działa teraz?

Odpowiedź: To zapis ostatniego potwierdzonego zastosowania. Bieżący stan trzeba odczytać także z Androida, uwzględniając Device Owner, enabled i Lock Task. Po błędzie stan lokalny i systemowy mogą się różnić.

Slajd 32 — Device Owner jest warunkiem ochrony

Cel: Rozdziel rolę zarządzania, konfigurację listy i uruchomienie blokady.

Powiedz:

> Device Owner daje aplikacji rolę potrzebną do zarządzania urządzeniem w przyjętym modelu. Widoczne wywołania ustawiają dozwolone pakiety i funkcje Lock Task. Sam tryb jest następnie uruchamiany i sprawdzany. To działanie przez oficjalne API, bez wymagania roota w tej implementacji. Zwykłe włączenie administratora urządzenia ani instalacja APK nie zastępują roli właściciela. Ważne jest również rozróżnienie stanu LOCKED od zwykłego przypięcia ekranu.

Wersja krótka:

> Device Owner umożliwia zarządzanie, a Lock Task ogranicza zadania według ustawionej listy. Sam APK tego nie uruchamia.

Przejście: Na tej podstawie aplikacja realizuje dwa sposoby pracy.

Profesor pyta: Czy bez Device Owner nie da się w ogóle zobaczyć pulpitu?

Odpowiedź: Pulpit i część zwykłego uruchamiania są dostępne. Brakuje jednak pełnej ochrony wymaganej przez pojedynczą sesję i chronioną stronę. Roli systemowej nie zastępuje sam interfejs.

Slajd 33 — Dwa tryby pracy

Cel: Pokaż różnicę zachowania przy powrocie.

Powiedz:

> W trybie wielu aplikacji użytkownik wraca do pulpitu i wybiera inny dozwolony program. W sesji jednej aplikacji środowisko zostaje zawężone, a powrót wymaga PIN-u. Decyzję przechowuje singleSession w regule konkretnego pakietu, więc różne kafelki mogą uruchamiać różne tryby. Pływające menu zapewnia dostęp do powrotu i administratora także nad innymi aplikacjami.

Wersja krótka:

> Wiele aplikacji pozwala przełączać się przez pulpit. Pojedyncza sesja wymaga PIN-u przed powrotem.

Przejście: Podobne tryby są dostępne dla wbudowanej strony internetowej.

Profesor pyta: Czy sesja pojedyncza zostawia na liście tylko jeden pakiet?

Odpowiedź: Dla aplikacji natywnej pozostawia własny pakiet Stanowiska i pakiet celu. Własny pakiet jest potrzebny do sterowania i autoryzacji. Dla strony wystarcza pakiet Stanowiska, bo WebActivity należy do tej aplikacji.

Slajd 34 — WebView ogranicza nawigację

Cel: Zdefiniuj origin i jasno nazwij zakres ograniczenia.

Powiedz:

> WebView wyświetla stronę wewnątrz aplikacji. Lista dozwolonych adresów jest porównywana jako origins, czyli schemat, host i port. Dla przykładu inna ścieżka w tym samym origin jest dozwolona, ale subdomena lub inny port wymagają dodatkowego wpisu. Administrator ustawia też JavaScript. Jest to kontrola nawigacji; nie przedstawiam tej listy jako zapory całego ruchu sieciowego urządzenia.

Wersja krótka:

> Lista originów kontroluje nawigację strony według schematu, hosta i portu. Nie jest globalnym filtrem całej sieci.

Przejście: Drugim ważnym obszarem kontroli jest dostęp administratora przez PIN.

Profesor pyta: Czy wpisanie example.org automatycznie dopuszcza wszystkie subdomeny?

Odpowiedź: Nie. Host jest porównywany dokładnie po normalizacji. Potrzebny jest pełny adres HTTP lub HTTPS, a dodatkowy host, np. logowania, trzeba wpisać osobno. Ścieżka nie zawęża originu.

Slajd 35 — PIN chroni dostęp administratora

Cel: Wyjaśnij role kolejnych mechanizmów kryptograficznych.

Powiedz:

> PIN nie jest zapisywany jako zwykły tekst. Z losowej soli i PIN-u wyprowadzana jest pochodna PBKDF2-HMAC-SHA256 z sześciuset tysiącami iteracji. Sól z pochodną jest dodatkowo szyfrowana AES-GCM, a kluczem zarządza Android Keystore. Kolejne błędne próby wywołują opóźnienia. Przy przygotowaniu trzeba zmienić domyślne 1111. Te mechanizmy podnoszą koszt zgadywania i chronią zapis, ale nie zastępują pełnej oceny bezpieczeństwa urządzenia.

Wersja krótka:

> PIN ma sól, pochodną PBKDF2, szyfrowany zapis i ograniczanie prób. Domyślny kod trzeba zmienić przed udostępnieniem stanowiska.

Przejście: Oprócz PIN-u sprawdzana jest poprawność danych konfiguracyjnych.

Profesor pyta: Czy PBKDF2 szyfruje PIN i czy klucz zawsze jest w StrongBox?

Odpowiedź: PBKDF2 wyprowadza pochodną, nie jest szyfrowaniem odwracalnym. Szyfrowanie zapisu realizuje AES-GCM. Kod używa Android Keystore, ale nie wymusza StrongBox ani nie potwierdza jego obecności na danym urządzeniu.

Slajd 36 — Błędna konfiguracja jest odrzucana

Cel: Przypisz każdemu warunkowi sens praktyczny.

Powiedz:

> Walidator odrzuca pustą listę, jeśli nie ma włączonej strony, a także duplikaty i nieobecne aplikacje. Nie pozwala ręcznie dodać własnego pakietu, ponieważ silnik dodaje go automatycznie. Osobna walidacja adresów akceptuje tylko poprawne origins HTTP lub HTTPS bez danych logowania w URL. Dzięki temu błędy wejścia mogą zostać wykryte przed próbą zastosowania konfiguracji w systemie.

Wersja krótka:

> Sprawdzane są pakiety, duplikaty, sens listy i poprawność URL. Własny pakiet dodaje silnik, a nie administrator.

Przejście: Poprawny zapis nadal trzeba odróżnić od polityki faktycznie zastosowanej.

Profesor pyta: Czy walidacja jest wykonywana tylko podczas zapisu?

Odpowiedź: Nie. Istotne kontrole są powtarzane przy zastosowaniu i uruchamianiu. Między zapisaniem konfiguracji a użyciem aplikacja może zniknąć lub zmienić wymagane uprawnienia.

Slajd 37 — Konfiguracja zapisana i zastosowana

Cel: Objaśnij odtwarzanie na konkretnym przykładzie rewizji.

Powiedz:

> Załóżmy, że rewizja siódma działa na urządzeniu, a administrator zapisuje ósmą. Dopóki jej nie zastosuje, przy odtwarzaniu obowiązuje ostatnia zastosowana lista, czyli siódma. Mechanizm restore uwzględnia także znacznik aktywnej sesji. Po restarcie BootReceiver może poprosić o otwarcie pulpitu, a MainActivity próbuje odtworzyć ochronę. To działanie zależy od stanu urządzenia, dlatego wymaga próby na docelowym sprzęcie.

Wersja krótka:

> Odtwarzana jest ostatnia zastosowana konfiguracja, a nie nowy, niezastosowany zapis. Stan po restarcie wymaga sprawdzenia na urządzeniu.

Przejście: Na tej podstawie mogę podsumować funkcje obecne w aplikacji.

Profesor pyta: Czy każda udana operacja restore ma osobny wpis w audycie?

Odpowiedź: Nie ma osobnego zdarzenia RESTORE_SUCCESS. Kod rejestruje żądanie z BootReceiver, błędy odtwarzania i wybrane zdarzenia Lock Task. Ogólny napis na slajdzie o audycie nie oznacza kompletnego zapisu każdego kroku przywracania.

Slajd 38 — Zakres obecny w aplikacji

Cel: Podsumuj rezultat z perspektywy obu użytkowników.

Powiedz:

> Użytkownik ma kontrolowany pulpit, wybrane aplikacje, pojedynczą sesję i stronę internetową. Administrator ma PIN, konfigurację, polityki, uprawnienia, ustawienia ekranu i audyt. Materiały instalacyjne wspierają przygotowanie urządzenia. Jest to podsumowanie funkcji obecnych w implementacji; następny krok to pokazanie ich zachowania w konkretnych scenariuszach.

Wersja krótka:

> Implementacja obejmuje pracę użytkownika i administrację, od pulpitu po polityki, PIN oraz lokalny audyt.

Przejście: Do takiego sprawdzenia przygotowano następującą demonstrację.

Profesor pyta: Czy ta lista jest protokołem odbioru funkcji?

Odpowiedź: Nie. Jest zestawieniem możliwości kodu. Protokół powinien dodatkowo zawierać warunki próby, urządzenie, wynik i ocenę każdego kryterium.

Slajd 39 — Demonstracja krok po kroku

Cel: Zapowiedz pokaz i nazwij obserwacje.

Powiedz:

> Demonstrację zaczynam od roli Device Owner i konfiguracji administratora. Następnie zapisuję i stosuję politykę, sprawdzam listę aplikacji oraz powrót z pojedynczej sesji. Dla WebView sprawdzam przejście do obcego originu. Na końcu oceniam restart i wyjście serwisowe. W każdym kroku liczy się wynik zaobserwowany na urządzeniu; sama obecność kroku na tej liście nie oznacza jeszcze udanego testu.

Wersja krótka:

> Pokaz obejmuje przygotowanie, zastosowanie polityki, kontrolę uruchamiania, PIN, WebView i odtworzenie po restarcie.

Przejście: Po demonstracji wrócę do mapy kodu odpowiadającego za te zachowania.

Profesor pyta: Jak wykonać próbę aplikacji niedozwolonej, jeśli nie ma jej kafelka?

Odpowiedź: Brak kafelka sprawdza tylko zachowanie interfejsu. Próbę ograniczeń systemowych trzeba wcześniej zaprojektować dla urządzenia, np. przez kontrolowany Intent do pakietu spoza listy. Wymaga to obserwacji systemu, a nie samego zdjęcia pulpitu.

Slajd 40 — Mapa modułów

Cel: Połącz zachowanie z miejscem implementacji.

Powiedz:

> Tabela wiąże elementy pokazu z odpowiedzialnościami w kodzie. Interfejs i ViewModel obsługują pracę ekranu, PolicyEngine wykonuje operacje polityki, a baza przechowuje stan. PIN, WebView, uprawnienia, restart i nakładka mają własne części. Nie będę odczytywał wszystkich nazw; ważne jest, że dla każdego zachowania można wskazać konkretny punkt implementacji.

Wersja krótka:

> Każdemu zachowaniu z pokazu odpowiada konkretna część kodu: interfejs, silnik, dane albo obsługa systemu.

Przejście: Taką mapę można następnie powiązać z wymaganiami funkcjonalnymi.

Profesor pyta: Czy autoryzacja administratora jest przechowywana w ViewModelu?

Odpowiedź: Właściwy termin autoryzacji adminUntil znajduje się w WorkspaceApp, a poświadczenie w PinStore. ViewModel i ekran korzystają z usług oraz prezentują stan; wiersz tabeli jest skrótem odpowiedzialności.

Slajd 41 — Wymagania mają pokrycie w kodzie

Cel: Wyjaśnij, co ta tabela potwierdza.

Powiedz:

> Każdemu wymaganiu przypisałem element implementacji. Na przykład listę opisują AppRule i PolicyRecord, a odtwarzanie realizują restore i BootReceiver. Takie powiązanie pomaga ocenić kompletność zakresu i znaleźć miejsce testu. Oznaczenie „w kodzie” potwierdza obecność rozwiązania, natomiast nie zastępuje funkcjonalnego odbioru na urządzeniu.

Wersja krótka:

> Tabela pokazuje śledzenie wymagań do kodu. Obecność implementacji to pierwszy etap, a wyniki prób są osobnym dowodem.

Przejście: Następnie wskażę pomiary potrzebne do oceny działania.

Profesor pyta: Czego brakuje do pełnej macierzy weryfikacji wymagań?

Odpowiedź: Identyfikatora testu, warunków i urządzenia, oczekiwanego wyniku, wyniku rzeczywistego oraz oceny. Samo wskazanie klasy nie potwierdza spełnienia całego wymagania.

Slajd 42 — Trzy pomiary do wykonania

Cel: Określ badane wielkości, nie wymyślaj wyników.

Powiedz:

> Plan obejmuje czas otwarcia pulpitu, czas zastosowania polityki i zachowanie po restarcie. Dla każdego pomiaru trzeba ustalić jednoznaczny początek i koniec oraz podać model i wersję systemu. Przykładowo zastosowanie liczę do potwierdzenia LOCKED i ukończenia operacji, a nie do samego naciśnięcia przycisku. Slajd nie zawiera jeszcze zebranych wyników tych pomiarów.

Wersja krótka:

> Do zbadania pozostają start, zastosowanie polityki i restart, według jawnie określonych warunków pomiaru.

Przejście: Na tej podstawie można uczciwie określić obecny stan weryfikacji.

Profesor pyta: Jaką średnią szybkość działania uzyskano?

Odpowiedź: Nie ma zebranej serii takich pomiarów. Nie podaję więc średniej. Najpierw trzeba określić liczbę prób, stan początkowy i sposób pomiaru, a potem zebrać wyniki.

Slajd 43 — Kod i dokumentacja to pierwszy etap

Cel: Wytłumacz znaczenie dwóch statusów w tabeli.

Powiedz:

> Tabela rozróżnia właściwości widoczne w konfiguracji i kodzie od zachowań wymagających prób. Można odczytać minimalne API, model lokalnych danych i podział rewizji. Odtwarzanie oraz skuteczność ochrony trzeba ocenić na urządzeniu. Dodatkowe budowanie i sprawdzenia logiki z dokumentacji poszerzają weryfikację, ale nie zmieniają automatycznie tych dwóch obszarów na przetestowane sprzętowo.

Wersja krótka:

> Stan „w kodzie” oznacza implementację, a „do prób” potrzebę potwierdzenia jej zachowania na sprzęcie.

Przejście: To samo rozróżnienie stosuję do pięciu tez pracy.

Profesor pyta: Czy 62 sprawdzenia z dokumentacji oznaczają 62 testy kiosku?

Odpowiedź: Nie. Obejmowały wybrane skompilowane funkcje pomocnicze, np. walidację, opóźnienia PIN-u i normalizację adresów. Nie uruchamiały DevicePolicyManager, nakładki ani całej sesji na fizycznym Androidzie.

Slajd 44 — Jak zweryfikować pięć tez

Cel: Dopasuj metodę sprawdzenia do każdej tezy.

Powiedz:

> Skuteczność ograniczeń sprawdzam próbami uruchomienia niedozwolonych komponentów i opuszczenia środowiska. Elastyczność oceniam przez konfigurację różnych scenariuszy. Użycie mechanizmów systemowych potwierdzam odczytem roli i stanu Lock Task. Teza o mniejszym nakładzie pracy wymaga porównania czasu przygotowania takiego samego stanowiska dwiema metodami. Sama wygoda formularza nie dowodzi jeszcze oszczędności czasu.

Wersja krótka:

> Każda teza ma własną metodę weryfikacji: próby ograniczeń, różne konfiguracje, odczyty systemu i porównanie czasu administracji.

Przejście: Przed badaniami trzeba poprawnie zainstalować i przygotować aplikację.

Profesor pyta: Jak zapewnić uczciwe porównanie czasu administracji?

Odpowiedź: Obie metody muszą prowadzić do tego samego stanu końcowego na porównywalnym urządzeniu. Trzeba oddzielić jednorazowe przygotowanie od późniejszych zmian i opisać doświadczenie osoby wykonującej zadanie.

Slajd 45 — Instalacja i przygotowanie urządzenia

Cel: Oddziel instalację APK, rolę właściciela i aktywację polityki.

Powiedz:

> ADB najpierw potwierdza dostęp do urządzenia. Instalacja z flagą r pozwala aktualizować zgodny pakiet z zachowaniem danych, a t dopuszcza testowy APK. Osobne polecenie nadaje Device Owner, jeśli Android akceptuje stan urządzenia. Potem trzeba zmienić PIN, przygotować uprawnienia i nakładkę, zapisać oraz zastosować politykę. Instalatory Bash i PowerShell automatyzują część tej procedury, ale nie obchodzą warunków systemu.

Wersja krótka:

> Instalacja APK, nadanie Device Owner i zastosowanie polityki to trzy różne etapy przygotowania urządzenia.

Przejście: Na końcu pozostają kierunki rozwoju po lokalnej implementacji.

Profesor pyta: Dlaczego aktualizacja z flagą r może zostać odrzucona?

Odpowiedź: Android sprawdza m.in. zgodność pakietu i podpisu. Nowy lokalny APK debug może mieć inny klucz niż wcześniejsza instalacja. Flaga r nie omija tej kontroli i nie jest gwarancją udanej aktualizacji.

Slajd 46 — Kolejne etapy rozwoju

Cel: Pokaż priorytety dalszych prac.

Powiedz:

> Najbliższy krok to pilotaż, próby migracji i odtwarzania oraz ocena użyteczności. Kolejne usprawnienia mogą obejmować import konfiguracji, szablony i procedurę odzyskiwania dostępu. Przed szerszą dystrybucją potrzebne są także podpis produkcyjny i przemyślany provisioning. Integracja z panelem centralnym przez API jest dalszym rozszerzeniem, którego obecna aplikacja lokalna nie implementuje.

Wersja krótka:

> Priorytetem są próby i przygotowanie dystrybucji. Import, QR oraz panel centralny to kolejne etapy rozwoju.

Przejście: Podsumuję teraz rezultat i jego najważniejsze znaczenie.

Profesor pyta: Które z tych rozszerzeń już działają?

Odpowiedź: Wymienione w kolumnie „kolejny etap” są propozycjami. Nie przedstawiam importu i eksportu, provisioningu QR ani centralnego API jako obecnych funkcji.

Slajd 47 — Kontrolowane środowisko i lokalna konfiguracja

Cel: Zamknij prezentację konkretnym rezultatem i następnym krokiem.

Powiedz:

> Rezultatem jest aplikacja łącząca kontrolowany pulpit, lokalną konfigurację i polityki Androida. Najważniejsze rozdzielenia to zapis i zastosowanie oraz zwykła praca i autoryzowana administracja. Implementacja daje podstawę do przygotowania różnych stanowisk, natomiast potwierdzenie skuteczności tez wymaga pilotażu i pomiarów na docelowym sprzęcie. To jest kolejny krok rozwoju projektu. Dziękuję za uwagę i jestem gotowy na pytania.

Wersja krótka:

> Powstała implementacja lokalnego stanowiska Android. Kolejny krok to pilotaż i pomiary potwierdzające jego działanie. Dziękuję za uwagę.

Przejście: Zatrzymaj slajd i poczekaj na pytania. Nie przełączaj już prezentacji.

Profesor pyta: Jaki jest najważniejszy wkład projektu?

Odpowiedź: Połączenie w jednej lokalnej aplikacji konfiguracji komponentów, dwóch trybów pracy, systemowych polityk, autoryzacji i trwałego stanu. Praktycznym celem jest przygotowanie urządzenia do określonego zadania w granicach mechanizmów Androida.

Demonstracja przy slajdzie 39

Wybierz sposób pokazania przed wystąpieniem

Jeśli tablet nie jest wcześniej przygotowany i sprawdzony, potraktuj slajd jako omówienie **procedury weryfikacji**. Pełne nadawanie Device Owner, pierwsze logowanie w zewnętrznej aplikacji czy dobieranie adresów SSO podczas prezentacji utrudnia ocenę właściwego mechanizmu. Na pokaz przygotuj dwa znane pakiety i kontrolowaną stronę bez rzeczywistych danych użytkowników.

Przy gotowym urządzeniu możesz powiedzieć: „Pokażę teraz kilka zachowań na tym konkretnym urządzeniu. Będę odróżniał oczekiwany efekt od wyniku, który rzeczywiście zaobserwujemy”.

Krok	Co robisz	Co mówisz i obserwujesz
Rola	Odczyt właściciela i stanu w panelu	„Najpierw potwierdzam przygotowanie Androida. Sama ikona aplikacji nie dowodzi roli właściciela”.
Administrator	Wejście PIN-em; kod początkowy zmieniony przed udostępnieniem	„To lokalna autoryzacja dostępu do ustawień. Nie jest to konto w centralnym systemie”.
Zapis	Zmiana listy i zapis konfiguracji	„Wzrosła rewizja zapisana. Aktywna lista nie musi jeszcze się zmienić”.
Zastosowanie	Naciśnięcie przycisku i kontrola wyniku	„Teraz porównuję rewizje oraz potwierdzenie systemowego Lock Task”.
Wieloaplikacyjność	Otwarcie A, powrót przez dom, otwarcie B	„Powrót do przygotowanego pulpitu jest dozwolony; ochrona pozostaje aktywna”.
Niedozwolony pakiet	Wcześniej przygotowana, kontrolowana próba uruchomienia poza listą	„To sprawdza ograniczenie systemu. Sam brak kafelka byłby jedynie sprawdzeniem interfejsu”.
Pojedyncza sesja	Uruchomienie odpowiedniej reguły i przycisk kłódki	„Powrót wymaga PIN-u. Po autoryzowanym zakończeniu wraca lista wielu aplikacji”.
WWW	Dozwolony adres i link do obcego originu	„Sprawdzam nawigację. Ten pokaz nie jest pomiarem całego ruchu sieciowego”.
Restart	Restart po przywróceniu chronionej konfiguracji	„Po odblokowaniu ekranu sprawdzam, czy wróciły zastosowana lista i stan ochrony”.
Serwis	Zakończenie sesji, autoryzacja, wyjście serwisowe	„Sprawdzam zniesienie ograniczeń. Device Owner pozostaje nadany”.

Pełny pokaz obejmuje oczekiwanie na system i interakcje, więc nie doliczaj go jako jednego krótkiego komentarza do slajdu. Przy małej ilości czasu wybierz zapis/zastosowanie, jeden powrót z sesji i WWW; pozostałe próby omów jako plan. Nie przedstawiaj pominiętych kroków jako zaliczonych.

Gdy pokaz nie powiedzie się

Jeśli brakuje Device Owner: „Urządzenie nie spełnia warunku rozpoczęcia pełnej ochrony. Nie będę nadawał

tej roli podczas wystąpienia. Pokażę sekwencję operacji i wrócę do przygotowania sprzętu przed właściwą próbą”.

Jeśli strona się nie otwiera: „Najpierw rozróżniam błąd sieci, certyfikatu i listy originów. Brak strony nie dowodzi, że polityka lokalnego pulpitu przestała działać”.

Jeśli zastosowanie zgłasza błąd: „To wynik tej próby. Sprawdzę komunikat, zapis błędu i stan Androida. Samo kliknięcie przycisku nie jest potwierdzeniem aktywacji”.

Jeśli nie masz urządzenia: „Ten slajd przedstawia przygotowany scenariusz, a nie wykonany dzisiaj test. Działanie systemowych ograniczeń pozostaje do potwierdzenia na sprzęcie”. Następnie przejdź do slajdu 40.

Doprecyzowania do trudniejszych slajdów

Slajd	Jak rozumieć skrót na slajdzie	Sformułowanie do zapamiętania
5 i 44	Tezy nie są jeszcze wynikami eksperymentu	„Implementacja uzasadnia sposób realizacji; skuteczność weryfikuję próbami”.
9	Lokalizacja pilotażu nie oznacza produkcyjnego wdrożenia	„To proponowane środowisko testowe”.
13	Kwota nie opisuje pełnego kosztu projektu	„Nie obejmuje urządzenia, pracy i utrzymania”.
15	Oceny ryzyka są jakościowe	„Nie są częstością awarii z próby statystycznej”.
21 i 40	Moduły na slajdzie to odpowiedzialności w jednym `:app`	„Podział logiczny, nie dziesięć modułów Gradle”.
22 i 27	Warunki uruchomienia są skrócone	„Mówię o chronionym przepływie; zwykły start wielu aplikacji jest możliwy bez pełnej ochrony”.
25	Opóźnienie PIN-u nie zaczyna się od pierwszego błędu	„Od piątej nieudanej próby”.
26	Kompensacja nie jest transakcją DPM i Room	„Kod próbuje odtworzyć poprzedni stan i zapisuje błędy”.
32	Device Owner umożliwia zarządzanie, ale nie wystarcza do ochrony	„Rolę trzeba uzupełnić zastosowaną polityką i stanem LOCKED”.
34	Origin nie jest samą domeną i nie filtruje wszystkiego	„Schemat, host, port; kontrola nawigacji”.
35	PBKDF2, AES-GCM i Keystore mają różne zadania	„Pochodna, szyfrowanie zapisu, zarządzanie kluczem”.
37	restore bierze zastosowane reguły; audyt nie zawiera osobnego sukcesu restore	„Ostatnia zastosowana konfiguracja, z kontrolą bieżącego systemu”.
39	Ukryty kafelek nie dowodzi systemowej blokady innego programu	„Potrzebna jest osobna próba uruchomienia spoza listy”.
41–43	Kod i test to różne rodzaje dowodów	„Jest implementacja; określona próba potwierdza określone zachowanie”.
45	Serwis nie usuwa zarządzania ani wcześniejszych decyzji uprawnień	„Ograniczenia stanowiska można wyłączyć, pozostawiając Device Owner”.
46	Roadmapa nie opisuje już dostępnych funkcji	„To propozycje kolejnych etapów”.

Ostatnia powtórka przed wystąpieniem

Wątki, do których możesz wrócić po pytaniu

1. **Device Owner → polityka → LOCKED.** Rola, konfiguracja i potwierdzone działanie to osobne elementy.
2. **save → apply.** Pierwsze zmienia zapis, drugie próbuje zmienić system i utrwalić wynik.
3. **revision → appliedRevision.** Potrafisz wyjaśnić przykład z rewizją 7 i 8.
4. **Wiele → jedna aplikacja.** Rozróżniasz powrót do pulpitu, koniec sesji, zamknięcie panelu i wyjście serwisowe.
5. **PIN.** PBKDF2 tworzy pochodną, AES-GCM szyfruje zapis, Keystore zarządza kluczem; od piątej porażki rośnie opóźnienie.
6. **Origin.** Znasz przykład innej ścieżki, subdomeny i portu oraz granicę kontroli nawigacji.
7. **Implementacja → weryfikacja.** Umiesz nazwać, co faktycznie sprawdzono i co wymaga sprzętu.

Jak ćwiczyć

Najpierw przejdź całą prezentację z wypowiedziami „Powiedz”, bez czytania pytań profesora. Zmierz czas. Jeśli przekracza limit, skracaj przede wszystkim powtarzające się slajdy: 6/20/21/40, 17/38/41 oraz 18/42/43. Zachowaj różnicę zapisu i zastosowania, wyjaśnienie Device Owner, dwa tryby, model danych, PIN i granice WebView.

Przy krótszej wersji nie pomijaj slajdów bez wyjaśnienia: użyj zdania „Wersja krótka” i przejdź dalej. Jeśli limit jest bardzo mały, ewentualny wybór mniejszej liczby slajdów ustal wcześniej. Ten scenariusz zachowuje wszystkie 47 w oryginalnej kolejności.

W drugiej próbie odpowiadaj na pytania po jednym zdaniu, a szczegóły dopowiadaj dopiero na prośbę. Gdy nie znasz wyniku, powiedz: „Tego nie potwierdziłem pomiarem. Mogę wskazać implementację i sposób sprawdzenia”. Nie zastępuj wyniku przypuszczeniem.

Przygotowanie na jutro

Na dziś wybierz jedną próbę trwającą około 45–60 minut. Przez pierwsze 10 minut powtórz siedem głównych rozróżnień powyżej. Następnie przejdź slajdy z wypowiedziami podstawowymi i stoperem, bez zatrzymywania się po każdym błędzie. Po tej próbie zaznacz najwyżej pięć miejsc, w których zabrakło Ci wyjaśnienia. Przez kolejne 10–15 minut popraw właśnie te fragmenty i odpowiedz na pytania do slajdów 22, 26, 31–35, 37 i 43. Na końcu powiedz otwarcie i zakończenie bez czytania.

Jutro przed wystąpieniem wystarczy krótka powtórka ze ściągi: kolejność tematów, przykład rewizji 7/8, różnica Device Owner/Lock Task, pochodna/szyfrowanie oraz wynik kodu/wynik próby. Sprawdź dostęp do prezentacji i notatek. Jeśli planujesz demonstrację, przygotuj urządzenie przed wejściem; jeśli go nie ma, użyj wariantu omówienia procedury ze slajdu 39.

Nie ucz się całego modułu słowo w słowo. Zapamiętaj sens, pierwsze zdanie każdego ważnego slajdu i drogę do następnego tematu. Gotowe wypowiedzi mają być podporą do ćwiczenia.

Nazwy, które warto wypowiedzieć na głos

Zrób krótką próbę z nazwami Device Owner, Lock Task, PolicyEngine, ViewModel, WebView, Room i Keystore. Nie musisz odczytywać pełnych pakietów Kotlin ani całych wywołań ze slajdu. PBKDF2 możesz przedstawić jako „funkcję wyprowadzania pochodnej PIN-u”, a potem dopiero podać nazwę algorytmu.

Krótkie otwarcie i zakończenie do nauczenia

Otwarcie: „Przedstawiam Stanowisko, czyli lokalnie konfigurowaną aplikację Android, która udostępnia wybrane funkcje urządzenia i korzysta z systemowych polityk do ograniczania środowiska pracy”.

Zakończenie: „Najważniejszym rezultatem jest połączenie pulpitu, trwałej konfiguracji i polityk Androida. Następnym krokiem są próby na docelowym urządzeniu, które pozwolą ocenić skuteczność rozwiązania i postawione tezy”.